

Practical No.1

Defination : Write a python program to demonstrate basic input and output operations.

Aim/Objective:

- Take input from user
- Display output using the **print()** function.
- To understand basic input and output operations in python.

Source Code:

```
#input
num=int(input("Enter value"))

#output
print(f"Value is {num}")
```

Enter value20

Value is 20

Conclusion:

The program demonstrate how Python uses input() to accept data from the user and print() to display the output on the screen.

Practical No.2

Defination:

Write a program that demonstrates the use of different data types and type casting.

Aim/Objective:

- To understand the use of different built in data types.
- To demonstrate the conversion of one data types to another data type using type casting.
- To understand the implicit(Automatic type changes based on data) and explicit (forcefully transition from one type to another type by user) type conversion.
- To observe the effect of type casting on programmer output.

Source Code:

```
integer =1;
String="Kaif"
```

Float=30.10
Boolean =True

```
print(f"Integer {integer}")
print(f"String {String}")
print(f"Float {Float}")
print(f"Boolean {Boolean}")
print(f"Before Type casting : {integer}")
print(f"Data Type : {type(integer)}")
integer = float(integer) # explicit type casting
print(f"After type casting : {integer}")
print(f"Data Type : {type(integer)}")
```

Output:

```
Integer 1
String Kaif
Float 30.1
Boolean True
Before Type casting : 1
Data Type : <class 'int'>
After type casting : 1.0
Data Type : <class 'float'>
```

Conclusion :

The program successfully demonstrate the use of different data types such as **integer,float,string,boolean** in Python.

Furthermore,it illustrates the type casting by converting values from one to another,Therefore we understand how python handles different types and their conversion.

Practical 3

Defination : Write a program to perform arithmetic,relational and logical operations using Python Operators

Aim/Objectcive:

- To understand the different Python Operators.
- To perform calculation and comparisons for understanding the use of operators.

Source Code:

```
num_1=int(input("Enter 1st Value "))
num_2=int(input("Enter 2nd Value "))

try:
    print("Arithmetic Operator ".center(50,'-'))
    print(f"Addition :{num_1+num_2}")
    print(f"Substraction :{num_1 + num_2}")
    print(f"Division : {num_1/num_2}")
    print(f"Multipulication : {num_1 * num_2}")
    print(f"Remainder of Division : {num_1 % num_2}")
except ZeroDivisionError as obj:
    print(obj)

print("Relational Operator".center(50,'-'))
print("\n")

print(f" Less than : {num_1 < num_2}")
print(f"Greater than : {num_1 > num_2}")
print(f"Lessthan equal to : {num_1 <=num_2}")
print(f"Greaterthan equal to : {num_1 >=num_2}")
print(f"Equal or Not : {num_1 == num_2}")
print("\n")

print("Logical Operator".center(50,'-'))

print('\n')

print(f" all conditions are true : {num_1 < num_2 and num_1< 0}")
print(f"Anyone condition are true :{num_1 < num_2 or num_1 < 0}")
```

Output:

```

Enter 1st Value 465
Enter 2nd Value 54
-----Arithmetic Operator -----
Addition :519
Substraction :411
Division : 8.611111111111111
Multipulication : 25110
Remainder of Division : 33
-----Relational Operator-----

Less than : False
Greater than : True
Lessthan equal to : False
Greaterthan equal to : True
Equal or Not : False

-----Logical Operator-----

all condition are true : False
Anyone condition are true :False

```

Conclusion :

The program successfully demonstrates the use of arithmetic, relational and logical operators in Python. It helps to understand how these operators are used to perform calculations, compare values, and evaluate logical conditions.

Practical 4

Definition: Write a python program to demonstrate string operation such as slicing, formatting and built-in string functions.

Aim/Objective:

- To understand the basic string operation.
- To understand the use of built-in functions.
- To perform string slicing.

Source Code:

```
string=input("Enter String Values : ")
print("String Slicing".center(50,'='))
slicing=string[1:len(string):2]# start from 2nd index to last index steps=2
print(slicing)

print("String Fromating".center(50,'='))

name='Kaif'
print('Hello' + name) # string concatenation

# f string (commonly used string formatting method
print(f'Hello {name}')

print("Built in string Functions ".center(50,'='))

#convert string into capital case
print(string.upper())

#convert into lower case
print(string.lower())

#1st letter will be capital
print(string.title())

# check, numerical value is present or not . it will return True or False.
print(string.isdigit())

#count frequency
print(string.count('i'))

#convert string into list
print(string.split(' '))
```

Output :

```
Enter String Values : kaif
=====String Slicing=====
```

```

af
=====String Formatting=====
HelloKaif
Hello Kaif
=====Built in string Functions =====
KAIF
kaif
Kaif
False
1
['kaif']

```

Conclusion:

The program successfully demonstrate the string slicing, formatting and built-in string functions in Python . It helps to understands how string can be manipulated and processed efficiently .

Practical 5.

Defination: Write a python program to create and manipulate lists using indexing,slicing and list comprehensions.

Aim/Objective:

- To create and access elements of a lists.
- To perform list indexing and slicing.
- Use list comprehensions for creating lists.

Source Code:

```

num_list=[20,40,50,120,160,200,220]
print("Find Value Using Index".center(50,'-'))
print(num_list[3])#index of collections are start from zero so value of 4th position will be printed

print("Slicing in list".center(50,'-'))
print(num_list[2:6:1]) # (start,stop,step) start from 3rd position to less than 6th position

print("List comprehension".center(50,'-'))

num_2=[14,15,19,16,18]
even_list=[x for x in num_2 if x%2==0]
print(even_list)

```

Output :

```
-----Find Value Using Index-----  
120  
-----Slicing in list-----  
[50, 120, 160, 200]  
-----List comprehension-----  
[14, 16, 18]
```

Conclusion:

The program successfully demonstrate the list creation and manipulation using indexing,slicing and list comprehensions in Python.

Practical 6.

Definition: Write a python program to demonstrate the use of tuples and sets with basic operations.

Aim/Objective:

- To create and access element of tuples
- To perform basic tuples operations.
- To create and manipulate sets.
- To understand basic set operations.

Source Code:

```
uidai_no=(123456123452,123456123452,545445636264)  
unique_no=set(uidai_no) # duplicate adhar will be removed  
  
uidai_no=unique_no  
print(uidai_no)  
  
print("Basic Operation In Tuple And Set".center(50,'-'))  
  
no=(1,2,3,5,8,9,7,6)  
# find index of target value  
print(no.index(8))
```

Output :

```
{545445636264, 123456123452}  
-----Basic Operation In Tuple And Set-----  
4
```

Conclusion:

The program successfully demonstrate the use of tuples and sets in Python and perform their basic operations.

Practical 7.

Definition : To write a python program to create a dictionary and demonstrate dictionary methods and iteration.

Aim/Objective:

- To create and access the a dictionary.
- To use basic dictionary methods.
- To iterate through dictionary elements.

Source Code:

```
students={101:"Kaif",102:"Aman",103:"Lalit",104:"Kartik"}
print("Iterate on Dictionary".center(50,'-'))
for id,name in students.items():
    print(f"ID : {id} Name : {name}")

print("Methods".center(50,'-'))

#assigning the copy of dictionary
#if any change happens in std_2 ,that will not affect on students
std_2=students.copy()
print("Copy of the students : ",std_2)

#Return Value of specific key

print(f"Name : {students.get(103)}")

#Update value:

students.update({102:"Shivam"})

#remove elements and return deleted element:

print(students.pop(102))
```

Output:

```
-----Iterate on Dictionary-----
ID : 101 Name : Kaif
ID : 102 Name : Aman
ID : 103 Name : Lalit
```


ID : 104 Name : Kartik

-----Methods-----

Copy of the students : {101: 'Kaif', 102: 'Aman', 103: 'Lalit', 104: 'Kartik'}

Name : Lalit

Shivam

Conclusion:

The program successfully demonstrates the creation, manipulation and iteration of a dictionary using Python dictionary methods.

Practical 8.

Definition: To write python program to demonstrate the mutable and immutable objects in Python.

Aim/Objective:

- To understand the mutable and immutable objects.
- To identify the difference between mutable and immutable objects .
- To observe how values can be changed in Python objects.

Source Code:

```
num=[12,13,14,15,16]
num_2=(20,30,40,50,60)
```

```
print("Mutable".center(50,'-'))
```

```
#it will shrink:
num.remove(15)
print(num)
```

```
#it will grow:
num.append(30)
print(num)
```

```
print("immutability".center(50,'-'))
```

```
try:
    #whenever i want change something after creating immutable object it will through error.
    #immutable object has fixed sized memory therefore it cannot be shrink,grow or update.
    num_2[5]=12
except Exception as ex:
    print(ex)
```

Output:

-----Mutable-----
[12, 13, 14, 16]
[12, 13, 14, 16, 30]
-----immutability-----
'tuple' object does not support item assignment

Conclusion:

The program successfully demonstrate the difference between mutable and immutable objects in Python and shows how their values behave when modified.

Practical 9.

Definition: To write a Python program to define use of user-defined function with different types of arguments.

Aim/Objective:

- To understand the concept of user-defined functions.
- To understand the use of different types of functions.

Source Code:

```
def calculate_area(length,width):  
    return length * width  
#default argument  
def greet_user(name,city='Rajkot'):  
    print(f"Name : {name} City : {city}")  
#dynamic parameter:  
def sum_all(*numbers):  
    return sum(numbers)  
  
print("Main".center(50,'-'))  
  
#Positional argument (Order matters)  
print("Positional Argument".center(50))  
  
area=calculate_area(5,3)  
print(f"Area of given value is : {area}")  
  
#Keyword argument (order doesn't matter):  
print("Keyword Argument".center(50))  
area=calculate_area(width=20,length=10)  
print("Default value")  
#greeting using default value:  
greet_user("Aman")
```

```
#overriding the default value:  
greet_user("Kaif","Saharsa")
```

```
#Dynamic Parameters:  
print("Dynamic Parameters".center(50))
```

```
result=sum_all(20,30,40,50)  
print(result)
```

Output:

```
-----Main-----  
          Positional Argument  
Area of given value is : 15  
          Keyword Argument  
Default value  
Name : Aman City : Rajkot  
Name : Kaif City : Saharsa  
          Dynamic Parameters  
140
```

Conclusion:

The program successfully demonstrates the use of user-defined functions with different types of arguments in python.

Practical 10.

Definition: Write a python program to demonstrate the recursive function using factorial or Fibonacci series.

Aim/Objective:

- To understand the concept of recursion.
- To implement the recursive function
- To calculate the factorial or Fibonacci series using recursion.

Source Code:

```
def facto(num):  
    if num == 1:
```

```
    return 1
    return num * facto(num-1)
print(facto(10))
```

Output:

3628800

Conclusion:

The program successfully demonstrates the use of recursion by calling a function repeatedly to calculate the factorial.